Bitácora de desarrollo del Backend

Feature 2: Análisis de impacto. AbastecePyme

Información general

Proyecto: AbastecePyme, API de dependencias de abastecimiento

Feature: Feature 2, Análisis de impacto (recorrido y consulta)

Tecnología: Python + FastAPI + Pydantic

Asistente de IA: Claude Code (Anthropic)

Rama de trabajo: feature/recorrido-impacto, creada a partir de feature/impacto-disponibilidad-miguel

Endpoint: GET /impacto/{elemento_id}

Registro de prompts

Prompt 1: Implementación del recorrido para el análisis de impacto

Herramienta: Claude Code

Prompt (versión redactada):

Implementa la Feature 2, Análisis de impacto. Antes de escribir código, revisa el repositorio para entender cómo están modelados los elementos (proveedores, insumos y productos) y sus relaciones, y respeta el lenguaje, la estructura de carpetas y el estilo existentes.

Contexto de negocio: cuando un elemento deja de estar disponible, producción necesita saber qué productos o procesos quedan afectados.

El grafo debe ser dirigido y tener una sola convención para las aristas: una arista que va de A hacia B significa “B depende de A” (si A falla, B queda afectado). No se debe corregir una dirección confusa con condicionales dentro del recorrido; si la dirección actual no permite ir de causa a consecuencia, se corrige el modelo, construyendo la lista de adyacencia en la dirección correcta. El grafo se representa con una lista de adyacencia (un diccionario donde cada nodo guarda su lista de vecinos).

El recorrido debe ser un BFS propio, sin librerías de grafos, siguiendo exactamente este esquema:

1. Crear una cola FIFO Q con el nodo inicial.

2. Crear la lista visitados con el nodo inicial.

3. Crear el diccionario padres con {nodo_inicial: None}.

4. Mientras la cola no esté vacía: sacar el primer elemento (actual) y, por cada vecino no visitado, agregarlo a visitados, registrar padres[vecino] = actual y encolarlo.

5. Retornar visitados y padres.

Ejemplo esperado: con aristas de 1 hacia 2, 6, 5, 4 y de 2 hacia 3, empezando en 1, el resultado es visitados = [1, 2, 6, 5, 4, 3] y padres = {1: None, 2: 1, 6: 1, 5: 1, 4: 1, 3: 2}.

La consulta debe diferenciar tres casos: elemento inexistente, elemento sin dependencias y elemento que afecta una cadena. La salida debe ser comprensible para producción: orden de visita por niveles, el camino por el que se llega a cada afectado (reconstruido con padres) y sin contar el nodo inicial como afectado. Agrega pruebas para el ejemplo anterior, elemento inexistente, elemento sin dependencias, una cadena de varios niveles y un grafo con ciclo.

Objetivo

Que, ante la indisponibilidad de un proveedor o insumo, la API identifique qué elementos de la red quedan afectados, en qué orden y por qué camino, usando un recorrido propio sobre el grafo de dependencias.

Requerimientos de diseño solicitados

Grafo dirigido con una única convención de aristas: una arista de A hacia B significa “B depende de A”.

Representación: lista de adyacencia (un diccionario donde cada nodo guarda su lista de vecinos).

Recorrido: BFS propio con cola deque, lista visitados y diccionario padres; sin librerías de grafos.

Dirección corregida en el modelo, no con condicionales dentro del recorrido.

Tres casos diferenciados: inexistente, sin dependencias y afecta una cadena.

Salida comprensible: orden de visita, nivel y camino de cada afectado.

Pruebas del ejemplo, los tres casos, una cadena de varios niveles y un grafo con ciclo.

Resultado obtenido

La rama de un compañero ya tenía la consulta GET /impacto/{id} con un recorrido provisional. Sobre esa base, Claude Code implementó el BFS y lo integró a la consulta, modificando estos archivos:

recorrido.py: lista de adyacencia de impacto, el BFS y la reconstrucción de caminos.

impacto.py: ahora usa el BFS y arma la respuesta con nivel y camino.

models.py: modelos de respuesta.

test_recorrido.py: pruebas automáticas.

README.md: explicación de qué significa “alcanzar”.

Ajustes durante el desarrollo

Después del prompt principal se hicieron estos ajustes, a partir de la revisión del equipo:

Ajuste	Motivo
Mostrar el camino y el nivel de cada afectado.	Que producción vea no solo qué se afecta, sino por qué cadena le llega la falla.
Quitar el recorrido provisional y el punto de extensión.	El BFS pasó a ser el recorrido definitivo; la consulta necesita también padres.
Simplificar la respuesta JSON.	La primera versión repetía la misma información y era difícil de consumir desde el frontend.
Dejar visitados como lista corta de ids.	Mostrar el resultado del BFS tal cual, en orden de visita, sin repetir datos.
Explicar “alcanzar” de forma sencilla en el README.	Que cualquier persona del negocio lo entienda con un ejemplo.

Explicación del código

La dirección de las flechas

El catálogo guarda cada relación como origen REQUIERE destino: “para producir pan necesito harina”. Esa flecha va de la consecuencia a la causa, y para saber qué se afecta cuando algo falla hay que ir al revés. En lugar de invertir la lógica con condicionales, se construye una lista de adyacencia nueva donde cada arista se voltea:

```
Catálogo: PRD-PAN requiere INS-HARINA, e INS-HARINA requiere PRV-MATERIAPRIMA
Impacto:  PRV-MATERIAPRIMA afecta a INS-HARINA, e INS-HARINA afecta a PRD-PAN

"Si falla el proveedor, falta harina; sin harina no hay pan."
```

recorrido.py: el recorrido

construir_adyacencia_impacto arma el diccionario donde cada nodo guarda sus vecinos, con la convención “una arista de A hacia B significa que B depende de A”.

bfs recorre en anchura: saca el primero de la cola, revisa sus vecinos y encola los que no ha visitado. Así avanza por niveles: primero lo afectado directamente, luego lo afectado a través de otro elemento. Como un nodo visitado no se vuelve a encolar, termina aunque haya ciclos.

reconstruir_camino parte de un afectado y sube por padres hasta el origen; al invertir esa lista se obtiene el camino de la falla.

```
def bfs(adyacencia, inicio):
    Q = deque([inicio])
    visitados = [inicio]
    padres = {inicio: None}
    while len(Q) != 0:
        actual = Q.popleft()
        for vecino in adyacencia.get(actual, []):
            if vecino not in visitados:
                visitados.append(vecino)
                padres[vecino] = actual
                Q.append(vecino)
    return visitados, padres
```

impacto.py: la consulta

Recibe el id del elemento que falla y decide cuál de los tres casos aplica:

Caso	Cuándo ocurre	Respuesta
inexistente	El id no está en el catálogo.	HTTP 404 con el mensaje.
sin_dependencias	Existe, pero nadie depende de él (p. ej. un producto terminado).	HTTP 200, afectados vacío.
afecta_cadena	El BFS alcanza otros elementos.	HTTP 200 con visitados y afectados.

Para cada afectado calcula el nivel (cuántas flechas hay desde el elemento que falló) y el camino (los ids desde el origen hasta él).

Ejemplo de respuesta: GET /impacto/PRV-QUIM

```
{
  "origen": { "id": "PRV-QUIM", "nombre": "Distribuidora de químicos",
              "capa": "proveedor" },
  "caso": "afecta_cadena",
  "mensaje": "Si 'Distribuidora de químicos' (PRV-QUIM) no está disponible,
              quedan comprometidos 3 elemento(s) de la red.",
  "visitados": ["PRV-QUIM", "INS-CONSERVANTE", "PRD-PONQUE", "PRD-TOSTADAS"],
  "afectados": [
    { "id": "INS-CONSERVANTE", "nombre": "Conservante alimentario",
      "capa": "insumo", "nivel": 1,
      "camino": ["PRV-QUIM", "INS-CONSERVANTE"] },
    { "id": "PRD-PONQUE", "nombre": "Ponqué artesanal",
      "capa": "producto", "nivel": 2,
      "camino": ["PRV-QUIM", "INS-CONSERVANTE", "PRD-PONQUE"] },
    { "id": "PRD-TOSTADAS", "nombre": "Tostadas integrales",
      "capa": "producto", "nivel": 2,
      "camino": ["PRV-QUIM", "INS-CONSERVANTE", "PRD-TOSTADAS"] }
  ]
}
```

models.py: la forma de la respuesta

NodoImpactado: un elemento de la red con id, nombre y capa (se usa para el origen).

NodoAfectado: un elemento al que le llega la falla; agrega nivel y camino.

ImpactoResponse: la respuesta completa con origen, caso, mensaje, visitados y afectados.

test_recorrido.py: las pruebas

Ocho pruebas automáticas, todas aprobadas:

El ejemplo del enunciado: visitados = [1, 2, 6, 5, 4, 3] y sus padres.

Un grafo con ciclo, para comprobar que el recorrido termina.

La reconstrucción de caminos y la dirección de la lista de adyacencia.

Los tres casos de la consulta: inexistente, sin dependencias y cadena de varios niveles.